

CONTEXTOS

Universal Context Layer for AI

Full Project Development Guide — Product + SRS + Architecture + Engineering Plan

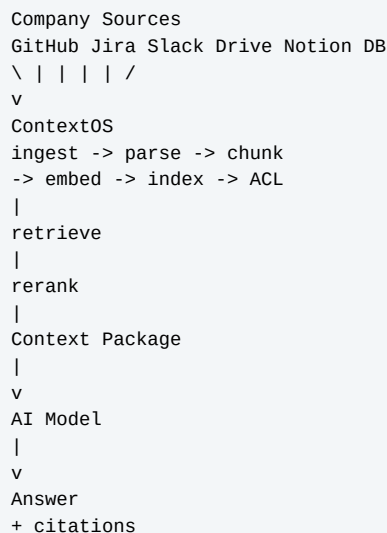
Core product: ContextOS connects an AI application to scattered organizational knowledge, retrieves the right authorized information, ranks it, assembles context, and returns grounded context with source references for an AI model.

Item	Specification
Product	ContextOS
Positioning	Enterprise context / knowledge infrastructure for AI
Primary problem	Company knowledge is fragmented across tools and databases
Primary value	Give AI the right authorized context at the right time
Backend	Java 21 + Spring Boot
Data	PostgreSQL + pgvector
Cache / queues	Redis + Kafka (V2)
Frontend	Next.js + TypeScript + Tailwind
Deployment	Docker → AWS

Target mindset: Build this like a senior software engineer, system designer, product thinker, security engineer and tester — not as a simple RAG demo.

1. Executive Summary

ContextOS is a platform that sits between enterprise data sources and AI applications. A company may have knowledge distributed across GitHub, Jira, Slack, Google Drive, Notion, databases, support systems and internal documents. ContextOS connects to those sources, ingests and indexes their content, applies permissions, retrieves relevant information for a user query, ranks the results, and produces a grounded context package for an AI model.



1.1 Product statement

“Connect your company’s knowledge once. ContextOS gives every AI application the relevant, permission-aware context it needs to produce grounded answers.”

1.2 What ContextOS is not

- Not simply a PDF chatbot.
- Not only a vector database.
- Not an LLM.
- Not a replacement for source systems.
- Not an excuse to dump an entire database into an LLM prompt.

2. The Real Problem

Problem	Example	ContextOS response
Fragmented knowledge	Issue in Jira, fix in GitHub, discussion in Slack	Retrieve related evidence across sources
Information overload	Thousands of documents	Semantic retrieval + ranking
Stale knowledge	Old document conflicts with current policy	Metadata, freshness and source priority
Permissions	Finance data visible only to Finance	ACL-aware retrieval
Poor AI grounding	AI answers from generic model knowledge	Return source-backed context
Integration burden	Every AI app builds its own connectors/RAG	Central context API

2.1 Real use cases

- Internal company AI assistant.
- Developer copilot over GitHub/Jira/technical documentation.
- Customer-support AI over CRM, tickets and product docs.
- Sales assistant over CRM, meetings and account history.

- HR assistant over approved company policies.
- Operations assistant over incidents, runbooks and dashboards.

3. Product Goals

- Create a unified context API for AI applications.
- Connect multiple enterprise knowledge sources.
- Normalize documents/events into a common internal representation.
- Support keyword + semantic retrieval.
- Enforce document-level/user-level permissions before context reaches the model.
- Return source metadata for explainability and citations.
- Measure retrieval quality, freshness and latency.
- Make connectors pluggable so new sources do not require rewriting the core.
- Build a clean MVP that can evolve toward enterprise Context-as-a-Service.

3.1 MVP scope

- Organization/user authentication.
- Connector framework.
- At least two source connectors; GitHub + local file/document ingestion are practical first choices.
- Document normalization and metadata.
- Chunking pipeline.
- Embeddings.
- PostgreSQL + pgvector semantic search.
- Keyword search using PostgreSQL full-text search.
- Hybrid retrieval.
- Basic reranking.
- RBAC/ACL filtering.
- Context API.
- Source citations.
- Redis caching.
- Admin/observability dashboard.
- Dockerized development.
- Automated tests.

3.2 V2/V3 scope

- Slack/Jira/Notion/Google Drive connectors.
- Incremental synchronization/webhooks.
- Kafka ingestion pipeline.
- Advanced reranking.
- Freshness-aware retrieval.
- Query rewriting.
- Conversation-aware retrieval.
- Permission synchronization.
- Evaluation datasets and retrieval benchmarks.
- Tenant-level quotas and billing.
- Enterprise SSO/audit.

- Knowledge graph/entity relationships.

3.3 Success metrics

Metric	Meaning
Recall@K	How often relevant evidence appears in top K.
Precision@K	How much of top K is actually relevant.
MRR	How early the first relevant result appears.
Grounded answer rate	Answers supported by retrieved evidence.
Citation correctness	Returned citations actually support the claim.
Freshness	How quickly source changes become searchable.
Retrieval latency	p50/p95 from query to context package.

4. Actors

Role	Capabilities
Platform Admin	Manage system connectors, policies and health.
Organization Admin	Manage sources, users, permissions and retrieval policies.
Developer	Use ContextOS API and inspect permitted sources.
End User	Ask questions through an AI application.
Connector Worker	Synchronize and process source content.

5. Core Data Lifecycle

```
SOURCE
|
CONNECT
|
FETCH / SYNC
|
NORMALIZE
|
PARSE
|
CHUNK
|
EMBED
|
INDEX
|
ACL / PERMISSIONS
|
QUERY
|
HYBRID RETRIEVAL
|
FILTER BY ACCESS
|
RERANK
|
CONTEXT ASSEMBLY
|
AI MODEL
|
GROUNDED ANSWER + SOURCES
```

6. Source Connector Architecture

Every external source should implement a common connector contract. The core retrieval system must not contain GitHub-, Jira- or Slack-specific logic.

```
public interface ContextConnector {

    ConnectorType type();

    List fetch(SourceCursor cursor);

    ConnectorHealth health();

    void validateConnection();

}
```

6.1 Connector responsibilities

- Authenticate with source.
- Fetch new/changed/deleted records.
- Convert source-specific objects into `SourceItem`.
- Expose source IDs and timestamps.
- Provide permission/ACL metadata where available.
- Support incremental sync after the initial import.

6.2 Normalized `SourceItem`

```
SourceItem
- id
- sourceType
- sourceUrl
- title
- content
```

- author
- createdAt
- updatedAt
- mimeType
- tenantId
- acl
- checksum
- metadata

6.3 Initial connector strategy

For a student-buildable MVP, start with GitHub + local document ingestion. Add Jira/Slack/Notion after the end-to-end retrieval pipeline is proven. This demonstrates connector abstraction without turning the project into an integration marathon.

7. Ingestion Pipeline

```
Connector
↓
Change detector
↓
Parser
↓
Cleaner
↓
Chunker
↓
Metadata enrichment
↓
Embedding service
↓
PostgreSQL / pgvector
```

7.1 Change detection

Use a stable source ID plus checksum/updated timestamp. If content has not changed, do not re-embed it. For deleted source records, mark or remove indexed chunks according to retention policy.

7.2 Chunking strategy

Start with structure-aware chunking: preserve headings, code blocks and document boundaries. Use a configurable chunk size and overlap rather than hard-coded magic numbers. For source-specific content, prefer semantic boundaries where practical.

8. Embeddings and Vector Search

An embedding converts text into a numerical vector representing semantic meaning. Similar queries and passages tend to be close in vector space.

```
Document chunk
↓
Embedding model
↓
[0.12, -0.08, 0.44, ...]
↓
pgvector

Query
↓
same embedding model
↓
query vector
↓
similarity search
```

8.1 Why pgvector?

It keeps relational metadata, tenant boundaries, permissions and vector search in one PostgreSQL system for the MVP. This reduces operational complexity compared with introducing a separate vector database too early.

8.2 Vector similarity

Cosine similarity is a practical default. The exact index and distance operator should be selected based on the embedding model and production scale.

9. Hybrid Search

Pure semantic search can miss exact identifiers such as ticket IDs, class names, error codes and product names. Pure keyword search can miss semantic meaning. ContextOS should combine both.

```
Query
|
+--> Keyword search -----+
| |
+--> Vector search -----+--> merge
|
rerank
|
top context
```

9.1 Example

Query: “Why did APOLLO-421 fail?” Keyword search is strong for the exact ticket ID. Semantic search can find the related GitHub PR and Slack discussion even when the exact phrase differs. Hybrid retrieval combines both signals.

9.2 Retrieval stages

```
Stage 1: retrieve top N keyword results
Stage 2: retrieve top N vector results
Stage 3: merge/deduplicate
Stage 4: apply authorization
Stage 5: rerank
Stage 6: select top K context chunks
```

10. Reranking

Initial retrieval should favor recall. Reranking then improves the order of candidate chunks based on query-document relevance.

MVP option

Use a deterministic weighted score combining semantic similarity, keyword relevance, freshness and source priority. This is transparent and easy to benchmark.

```
rerankScore =  
0.55 * semanticScore  
+ 0.25 * keywordScore  
+ 0.10 * freshnessScore  
+ 0.10 * sourcePriority
```

V2 option

Use a dedicated reranker model or LLM-based evaluator for difficult queries, while measuring the added latency and cost.

11. Permissions and Security — Critical

The most important enterprise requirement is: never retrieve data a user is not allowed to see. Filtering only after sending unauthorized context to an AI model is too late.

```
User  
↓  
Identity / tenant  
↓  
Permission context  
↓  
Retrieve candidates  
↓  
ACL filter  
↓  
Rerank authorized candidates  
↓  
Context package  
↓  
LLM
```

11.1 ACL model

Field	Example
tenantId	org_123
visibility	PRIVATE / TEAM / PUBLIC
allowedUsers	user IDs
allowedGroups	engineering
source permissions	inherited from GitHub/Jira/etc.

11.2 Security rules

- Every indexed item belongs to a tenant.
- Authenticated identity determines tenant; never trust tenantId from the request body.
- Authorization is enforced at retrieval time.
- Admin APIs use RBAC.
- Connector credentials are stored as secrets.
- Audit connector changes and permission changes.
- Prompt/response logging is configurable and redacted.

- Do not expose source content to users without permission.

12. Context Assembly

Context assembly converts ranked chunks into a compact, ordered evidence package for the downstream AI model.

```
ContextPackage
- query
- generatedAt
- sources[]
- title
- url
- sourceType
- excerpt
- relevance
- updatedAt
- instructions
- tokenEstimate
- permissionTrace
```

12.1 Context budget

Do not send every retrieved chunk. Set a token budget and select the highest-value non-duplicate evidence. Context assembly should prioritize diversity and direct relevance.

13. Context API

```
POST /api/v1/context/search
Authorization: Bearer

{
  "query": "Why did the Apollo payment release fail?",
  "topK": 8,
  "filters": {
    "sourceTypes": ["GITHUB", "JIRA", "SLACK"]
  }
}

Response:
{
  "requestId": "ctx_123",
  "results": [
    {
      "title": "APOLLO-421",
      "sourceType": "JIRA",
      "url": "...",
      "excerpt": "...",
      "score": 0.93,
      "updatedAt": "..."
    }
  ],
  "context": "...",
  "retrieval": {
    "keywordCount": 5,
    "vectorCount": 8,
    "rerankedCount": 8
  }
}
```

13.1 Admin APIs

Method	Endpoint	Purpose
POST	/api/v1/admin/connectors	Create connector.
GET	/api/v1/admin/connectors	List connectors.
POST	/api/v1/admin/connectors/{id}/sync	Start sync.
GET	/api/v1/admin/connectors/{id}/health	Connector health.
GET	/api/v1/admin/documents	Inspect indexed documents.
GET	/api/v1/admin/analytics/retrieval	Retrieval metrics.
POST	/api/v1/admin/policies	Configure retrieval policies.

14. Database Design

Table	Important fields	Purpose
users	id, tenant_id, role, status	Identity.
organizations	id, name, plan	Tenant.
api_keys	id, tenant_id, key_hash, status	API access.
connectors	id, tenant_id, type, status, config_ref	External source connections.
documents	id, tenant_id, connector_id, source_id, title, url, checksum, updated_at	Normalized source items.
chunks	id, document_id, chunk_index, content, embedding, token_count	Retrieval units.
acl_entries	chunk/document id, subject_type, subject_id, permission	Access control.
sync_runs	id, connector_id, started_at, completed_at, status, items_changed	Ingestion history.
retrieval_requests	id, tenant_id, request_id, query, latency_ms, top_k	Query telemetry.
retrieval_results	id, request_id, chunk_id, stage, score, rank	Retrieval trace.

Table	Important fields	Purpose
audit_logs	id, actor_id, action, resource, created_at	Administrative audit.

14.1 Vector column

The chunks table stores the embedding vector using pgvector. The vector dimension must match the selected embedding model and should be configurable at the application level.

15. Java / Spring Boot Architecture

```
com.contextos
  ■■■ auth
  ■■■ tenant
  ■■■ api
  ■ ■■■ controller
  ■ ■■■ dto
  ■ ■■■ exception
  ■■■ connector
  ■ ■■■ ContextConnector
  ■ ■■■ github
  ■ ■■■ local
  ■■■ ingestion
  ■ ■■■ parser
  ■ ■■■ chunker
  ■ ■■■ embedding
  ■ ■■■ sync
  ■■■ retrieval
  ■ ■■■ keyword
  ■ ■■■ vector
  ■ ■■■ hybrid
  ■ ■■■ reranker
  ■ ■■■ context
  ■■■ security
  ■ ■■■ acl
  ■ ■■■ policy
  ■■■ analytics
  ■■■ audit
  ■■■ common
```

15.1 Important design patterns

Pattern	Use
Adapter	External source connectors.
Strategy	Different retrieval/reranking policies.
Factory	Connector instantiation.
Pipeline	Ingestion stages.
Repository	Database access.
Specification	Permission/filter predicates.
Outbox/Event	Reliable async sync events in V2.

16. Redis

- Cache repeated context queries with tenant/user-aware keys.
- Cache connector health.
- Cache frequently used retrieval metadata.
- Rate-limit API consumers.

```
context::
connector-health:
rate::
```

Never use a cache key that can allow one user's authorized result to be returned to another user.

17. Kafka — V2

Kafka is useful when ingestion becomes asynchronous and continuous. Do not add it just to make the architecture look complex.

```
connector.sync.requested
document.changed
document.deleted
```

```
embedding.created  
retrieval.completed
```

Connector

```
-> Kafka  
-> ingestion workers  
-> chunk/embed/index
```

18. Ingestion at Scale

A mature architecture separates user query latency from ingestion work. A sync should not block the API request thread while thousands of documents are parsed and embedded.



19. Freshness Strategy

Approach	Use
Manual sync	MVP / low-frequency sources.
Scheduled polling	Sources without webhooks.
Webhook/event	Near-real-time updates.
Checksum	Avoid reprocessing unchanged content.
Updated timestamp	Incremental source query when supported.

20. Observability

A senior implementation should be able to answer not only “What did we retrieve?” but “Why did we retrieve it, how long did it take, and was the user allowed to see it?”

Metric	Example
Ingestion throughput	chunks/minute
Sync latency	source change → searchable
Embedding latency	ms/chunk
Retrieval p50/p95	120 / 380 ms
Recall@K	0.82
MRR	0.76
Unauthorized filtered	count
Cache hit rate	31%
Index size	documents/chunks/vectors

20.1 Request tracing

Every query should have a request ID. Record stages such as keyword retrieval, vector retrieval, authorization filtering, reranking and context assembly.

21. Testing Strategy — Treat Retrieval as a Product Feature

A RAG system cannot be considered correct merely because an API returns HTTP 200. You must test retrieval relevance, permissions, freshness and grounding.

Test layer	What to test
Unit	Chunking, scoring, ACL rules, query filters, context assembly.
Integration	PostgreSQL + pgvector + Redis.
Connector contract	Normalized SourceItem and sync behavior.
Security	Cross-tenant access, private documents, revoked permissions.
Retrieval evaluation	Recall@K, Precision@K, MRR, nDCG where applicable.
Freshness	Updated/deleted source appears/disappears correctly.
E2E	Source → ingest → index → query → context.
Load	Concurrent searches and ingestion workload.
Failure	Connector outage, embedding failure, DB/Redis degradation.

21.1 Build an evaluation dataset

Create 30–100 representative questions with expected relevant source documents. Example: query, expected document IDs, acceptable top-K threshold. This lets you measure changes instead of relying on screenshots or intuition.

21.2 Golden test example

Question:
"Why was Apollo payment release delayed?"

Expected:
JIRA APOLLO-421
GitHub PR #981
Slack incident discussion

Pass:
At least 2 expected sources in top 5
AND
no unauthorized source returned

22. Failure Modes

Failure	Expected behavior
Connector unavailable	Mark connector unhealthy; keep existing index; retry later.
Embedding service fails	Queue failed items; do not corrupt existing index.
Document deleted	Remove/soft-delete its chunks from retrieval.
Permission revoked	Retrieval must stop returning the content immediately or within defined propagation SLA.
Vector search unavailable	Optionally fall back to keyword search if policy allows.
Redis unavailable	Core retrieval continues without cache where safe.
No relevant context	Return empty/low-confidence context; downstream AI should be instructed not to invent evidence.

23. Product Dashboard

Screen	What to show
Overview	Documents, chunks, connectors, sync health, query volume, latency.
Connectors	Connection status, last sync, records changed, errors.
Knowledge Explorer	Documents, source URL, updated time, ACL summary.
Retrieval Explorer	Query, retrieved chunks, scores, filtering, final context.
Evaluation	Recall@K, MRR, test-set results and regressions.
Users & Roles	Tenant users, roles, permissions.
Audit	Connector, permission and policy changes.

24. UX Principle

The dashboard should make the system explainable. For any query, a developer should be able to inspect: which documents were found, why they ranked highly, which were removed by permissions, and which sources were finally sent to the AI model.

25. Security Architecture

```
API Key / JWT
↓
Identity
↓
Tenant resolution
↓
RBAC
↓
Query validation
↓
ACL-aware retrieval
↓
Context package
↓
AI application
```

25.1 Threats to consider

- Cross-tenant data leakage.
- Permission changes not propagating.
- Connector token theft.
- Prompt injection inside retrieved documents.
- Malicious documents attempting to override system instructions.
- Sensitive content in logs.
- Overly broad connector scopes.

25.2 Prompt injection defense

Retrieved documents are data, not instructions. The downstream AI prompt should explicitly distinguish system instructions from retrieved source content. ContextOS should preserve source boundaries and metadata instead of concatenating everything as trusted text.

26. Performance Design

- Use indexes for tenant/source/time queries.
- Use pgvector indexes when dataset size justifies them.
- Batch embeddings during ingestion.

- Cache repeated queries.
- Limit candidate counts before reranking.
- Avoid fetching full document bodies during retrieval if excerpts are enough.
- Paginate admin explorer endpoints.
- Move heavy ingestion to workers as scale increases.

27. Full Development Roadmap

Develop in vertical slices. Every phase ends with a runnable and testable product.

Phase	Work	Deliverable
0 — Product discovery	Use cases, personas, data-flow, threat model, SRS	Approved product blueprint
1 — Foundation	Java 21, Spring Boot, PostgreSQL, Docker	Running backend
2 — Auth/Tenant	Users, organizations, API keys, RBAC	Secure multi-tenant API
3 — Connector core	ContextConnector + local source + GitHub	Normalized source ingestion
4 — Ingestion	Parsing, chunking, checksums, metadata	Indexed chunks
5 — Embeddings	Embedding adapter + pgvector	Semantic retrieval
6 — Hybrid retrieval	Keyword + vector + merge + ACL	Useful context API
7 — Reranking	Deterministic scoring + freshness/source priority	Better top-K
8 — Context assembly	Token budget, dedup, citations	LLM-ready context
9 — Redis	Cache + rate limiting + metadata	Lower latency
10 — Dashboard	Connectors, explorer, retrieval trace, evaluation	Product UI
11 — Testing	Golden dataset, security, integration, load	Quality baseline
12 — Cloud	AWS, CI/CD, monitoring	Public demo
13 — V2	Kafka, incremental sync, more connectors, advanced reranking	Scalable product

28. Six-Week Build Plan

Week	Focus
1	Architecture, Spring Boot, DB, tenant/auth/RBAC, Docker.
2	Connector abstraction + local/GitHub ingestion + normalization.
3	Chunking + embeddings + pgvector + vector search.
4	Hybrid retrieval + ACL + reranking + context assembly.
5	Redis + dashboard + retrieval trace + evaluation dataset.
6	Security testing + load/failure tests + AWS + CI/CD + demo + documentation.

29. Vibe-Coding Strategy

- Generate boilerplate with AI; own the architecture yourself.
- Ask for one bounded feature at a time.
- Give the AI your package structure, entity schema and API contract.
- Never allow generated code to bypass authorization.
- Generate tests together with each service.
- Use mock connectors/providers before real credentials.
- Review SQL, transaction boundaries and vector queries manually.
- Keep a decision log explaining why each technology exists.
- Benchmark retrieval before claiming it is “smart.”

30. Senior Software Engineer Checklist

- Clear module boundaries.
- Interface-based connectors.
- DTO/entity separation.
- Centralized exception handling.
- Validation at API boundaries.
- Configuration externalized.
- Database indexes considered.
- Idempotent ingestion.
- Explicit transaction boundaries.
- Structured logging and request IDs.
- Backward-compatible API versioning.
- Clean Git history and documentation.

31. Senior System Designer Checklist

- Tenant isolation.
- Separation of query path and ingestion path.
- Synchronous retrieval vs asynchronous ingestion trade-off.
- Cache failure strategy.
- Connector outage strategy.
- Scaling strategy.
- Data retention strategy.
- Freshness SLA.
- Permission propagation model.
- Observability and capacity planning.

32. Security Engineer Checklist

- Least-privilege connector scopes.
- Secrets outside source code.
- API key hashing.
- RBAC.
- ACL-aware retrieval.
- Audit logs.
- PII/log redaction.
- Prompt-injection boundaries.
- Cross-tenant tests.
- Permission-revocation tests.

33. Tester / QA Checklist

- Golden retrieval dataset.
- Recall@K and MRR baseline.

- Negative authorization tests.
- Deleted-document tests.
- Updated-document tests.
- Connector retry tests.
- Embedding failure tests.
- Concurrent query tests.
- Load tests.
- End-to-end source-to-answer tests.

34. Product / Entrepreneur Checklist

- Who pays? AI SaaS/enterprise teams.
- What do they buy? Context API + connectors + retrieval + governance.
- What is measurable? Retrieval quality, latency, freshness and grounded-answer rate.
- What creates stickiness? Connectors, permissions, indexed knowledge and evaluation history.
- What becomes the moat? High-quality retrieval/evaluation + permission-aware enterprise context + integrations.
- What should not be built first? 20 connectors, Kubernetes, complex agents, proprietary LLM.

35. Demo Story

1. Connect GitHub + local company docs.
2. Run sync.
3. Show documents/chunks indexed.
4. Ask:
"Why did Apollo payment release fail?"
5. Show hybrid retrieval results.
6. Show one result removed because user lacks access.
7. Show final ranked context with citations.
8. Send context to an AI model.
9. Show grounded answer.
10. Modify a source document.
11. Re-sync.
12. Ask again and demonstrate freshness.

36. Product Evolution

```
MVP
Context API
+ local/GitHub connectors
+ chunking
+ embeddings
+ pgvector
+ hybrid search
+ ACL
+ citations
+ dashboard
|
v
V2
+ Jira / Slack / Notion / Drive
+ incremental sync
+ Kafka
+ advanced reranking
+ evaluation framework
|
v
V3
+ query rewriting
+ conversation memory
+ knowledge graph
+ enterprise SSO
+ governance
+ billing
|
v
Context Infrastructure Platform
+ Context-as-a-Service
+ permission-aware AI data layer
+ reusable enterprise knowledge API
```

37. Final Recommended Technology Stack

Layer	Technology	Why
Language	Java 21	Primary career target; strong backend ecosystem.
Backend	Spring Boot	REST, security, DI, persistence, observability.
Security	Spring Security + API keys/JWT	Auth/RBAC.
Database	PostgreSQL	Relational metadata, tenants, ACLs.
Vector	pgvector	Semantic retrieval without another DB in MVP.
Cache	Redis	Query cache, rate limiting, health metadata.
Messaging	Kafka — V2	Async ingestion/events.
Frontend	Next.js + TypeScript + Tailwind	Operational dashboard.
Testing	JUnit 5, Mockito, Testcontainers	Backend/integration tests.
Evaluation	Custom golden dataset + retrieval metrics	Prove retrieval quality.
DevOps	Docker + GitHub Actions	Repeatable builds/CI.
Cloud	AWS	Deployment and cloud evidence.

38. Resume Positioning

ContextOS — Permission-Aware Universal Context Layer for AI

- Built a Spring Boot context platform that ingests enterprise knowledge, performs hybrid semantic/keyword retrieval with pgvector, and returns permission-aware source-backed context through a unified API.
- Designed a pluggable connector architecture and ingestion pipeline for normalization, chunking, embeddings, incremental synchronization and source metadata.

- Implemented tenant-aware ACL filtering, Redis caching and retrieval tracing to provide secure, explainable context for downstream AI applications.

Use real benchmark numbers only after testing: retrieval latency, Recall@K, MRR, indexed documents, sync freshness, cache hit rate, etc.

39. Interview Questions to Master

- Why does an AI application need ContextOS?
- Why vector search?
- Why hybrid search?
- Why pgvector instead of a dedicated vector DB?
- How do embeddings work?
- How do you chunk documents?
- How do you prevent unauthorized information from reaching the LLM?
- How do you handle permission changes?
- How do you detect document updates?
- How do you handle deletion?
- Why separate ingestion from retrieval?
- Why Kafka in V2?
- How do you measure retrieval quality?
- What is Recall@K? MRR?
- How would you scale to 100 million chunks?
- How do you defend against prompt injection in retrieved documents?
- How do you add a new connector without modifying the retrieval engine?

40. Final Build Principle

The impressive version of ContextOS is not the one with the largest number of connectors or the fanciest AI. It is the one that can prove four things: the right information was retrieved, the wrong information was excluded, the context is fresh and the final answer is grounded in traceable sources.

Build the MVP end-to-end first. Then measure retrieval quality. Then harden permissions and failure modes. Finally add scale, connectors and advanced intelligence.

END OF CONTEXTOS DEVELOPMENT GUIDE — VERSION 1.0